

Mini-dossier de Conception : Fintech-IAM

1. Cadrage

- **Problème résolu** : L'application répond au besoin de gestion automatisée et sécurisée des tontines numériques. Elle assure le suivi rigoureux des cycles d'épargne, la transparence des dépôts, la répartition automatique des commissions et le calcul fiable des montants retirables par les clients.
- **Utilisateurs** : Les agents de terrain (qui saisissent les opérations) et l'institution (qui supervise les flux financiers).
- **Opérations incluses** : Création de clients, ouverture de cycles, enregistrement de dépôts, clôture automatique, calcul des commissions (Agent/Institution) et retrait global.
- **Technologie** : J'ai choisi **Spring Boot (Java)** avec **PostgreSQL**. Cette stack est idéale pour une approche *Spec-Driven* car elle permet de séparer strictement la logique métier (couche Service) des interfaces (Couche Controller), tout en garantissant la cohérence des données via les transactions.

2. Règles métier

Mon application respecte les règles suivantes :

- **Client** : Un client est obligatoirement lié à un agent.
- **Mise** : Doit être strictement positive et multiple de 100 FCFA.
- **Cycle** : Limité à 31 collectes. Il démarre avec le statut `EN_COURS`.
- **Clôture** : Automatique dès la 31ème collecte. La retenue (égale à une mise) est partagée à 50% pour l'agent et 50% pour l'institution.
- **Calcul financier** : Le solde retirable n'est pas stocké en dur, il est recalculé dynamiquement à partir des mouvements (`CREDIT_CLIENT - RETRAIT`).
- **Intégrité** : Les opérations sont transactionnelles. Aucun dépôt n'est possible sur un cycle clôturé.

3. Modèle de données

Le modèle a été conçu pour garantir une traçabilité totale via une approche événementielle (les mouvements).

- **Agent** : id, username, nom, prénom, email.
- **Client** : id, nom, téléphone, @ManyToOne(Agent).
- **Cycle** : id, mise, nbCollectes, status, dateOuverture, @ManyToOne(Client).
- **Mouvement** : id, type (MISE, RETENUE, COM_AGENT, COM_INSTITUTION, CREDIT_CLIENT, RETRAIT), montant, date, @ManyToOne(Cycle), @ManyToOne(Client).

4. Traçabilité des règles métier

Règle métier	Implémentation (Classe/Méthode)	Test prévu
Multiple de 100 FCFA	<code>FintechService.createCycle()</code>	Mise 1050 refusée
Max 31 collectes	<code>FintechService.createDepot()</code>	Dépôt au-delà de 31 refusé
Clôture automatique	<code>FintechService.closeCycle()</code>	Vérif. statut CLOTURE à 31
Calcul retirable	<code>FintechService.getMontantRetirable()</code>	Somme mouvements CREDIT/RETRAIT

5. Conclusion

Cette solution respecte les principes *Spec-Driven* : toute règle métier est isolée dans la couche `Service` et testée indépendamment. La base de données est protégée par des transactions, assurant qu'aucun calcul financier ne puisse être compromis en cas d'erreur lors de l'exécution.